Applied Economics

Introduction

Pello Aspuru

Winter 2026/2027

Welcome to Applied Economics

All course material (including these slides) will be distributed via **moodle**.

Prior knowledge at the level of Introductory Econometrics (BA) will be assumed throughout the course.

Class participation is highly encouraged, determines 10% of final grade.

Office hours: by email appointment via pello.aspuru@univie.ac.at

People who signed up online for this class and signed the list today are considered **enrolled**.

If you are registered, you will receive a grade!

You can de-register until **October 8th** (after lecture 2).

Welcome to Applied Economics

All course material (including these slides) will be distributed via **moodle**.

Prior knowledge at the level of Introductory Econometrics (BA) will be assumed throughout the course.

Class participation is highly encouraged, determines 10% of final grade.

Office hours: by email appointment via pello.aspuru@univie.ac.at

People who signed up online for this class and signed the list today are considered **enrolled**.

If you are registered, you will receive a grade!

You can de-register until **October 8th** (after lecture 2).

Course Overview

In this Course You Will Learn to...

- Use econometric tools to produce empirical analyses.
- Learn to interpret, and critically evaluate existing empirical studies.
- Understand possibilities and limitations of empirical analyses.

Course Tools

Statistical Software: R and RStudio

- Freely available open source software
- Active user community providing up-to-date functions & support
- Extensive statistical modeling capabilities
- Relevant for industry jobs

Auxiliary Software

- Large Language Models (ChatGPT, Claude etc.) to support learning & coding.
- Latex for drafting academic-style documents, problem set.

Course Tools

Statistical Software: R and RStudio

- Freely available open source software
- Active user community providing up-to-date functions & support
- Extensive statistical modeling capabilities
- Relevant for industry jobs

Auxiliary Software

- Large Language Models (ChatGPT, Claude etc.) to support learning & coding.
- Latex for drafting academic-style documents, problem set.

Course Assessment

Class Participation (10%)

Classroom engagement through questions & comments is encouraged!

One Problem Set (25%)

- Work on problem sets in groups of up to 3 students, fill in names here.
- First presentation date: December 2nd/3rd 2026.
- Second presentation date: January 13th/14th 2027.

Midterm Exam (25%)

Preliminary date: November 25th/26th 2026.

Final exam (40%)

Date: Wednesday, 27th January 2027.

Course Assessment

Class Participation (10%)

Classroom engagement through questions & comments is encouraged!

One Problem Set (25%)

- Work on problem sets in groups of up to 3 students, fill in names here.
- First presentation date: December 2nd/3rd 2026.
- Second presentation date: January 13th/14th 2027.

Midterm Exam (25%)

Preliminary date: November 25th/26th 2026.

Final exam (40%)

Date: Wednesday, 27th January 2027.

Useful Textbooks

1. **Heiss, F. (2020).** Using R for Introductory Econometrics (2nd edition)
   available online: http://urfie.net/readOnline.html
2. **Hanck, C., Arnold, M., Gerber, A., & Schmelzer, M. (2023).** Introduction to Econometrics with R
   available online: https://www.econometrics-with-r.org/index.html
3. **Oswald, F., Viers, V., Villedieu, P., & Kennedy, G. (2020).** Introduction to Econometrics with R. Sciences Po, Department of Economics, Paris, France.
   available online: https://scpoecon.github.io/ScPoEconometrics/.

If you need to read up on theoretical concepts:

- Angrist, J. D., & Pischke, J.-S. (2008). Mostly Harmless Econometrics.
- Wooldridge, J. (2015). Introductory Econometrics: A Modern Approach.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data → Tidy dataset.
2. Find patterns in the data → Data Visualization and Descriptive Statistics.
3. Apply econometric tools → Establish causality and estimate magnitude.

What is Applied Economics?

The use of economic theory and **data** to answer real-world **economic** questions.

- Test what economic theory predicts.
  - Theory: a higher minimum wage reduces employment. Do we see this in the data?
- Discover (causal) relationships between economic variables and quantify them.
  - Does immigration increase unemployment for locals? By how much?
- Evaluate and design policies that achieve the intended objective at the **lowest possible cost.**
  - How much money do we need to subsidize public transport to save 1 ton of carbon emissions?

1. Gather, clean and combine multiple sources of data $\rightarrow$ Tidy dataset.
2. Find patterns in the data $\rightarrow$ Data Visualization and Descriptive Statistics.
3. Apply econometric tools $\rightarrow$ Establish causality and estimate magnitude.

What is R ?

- **Programming Language** A formal system of rules and symbols that allows humans to write instructions a computer can understand and execute.
  - Perfect replicability.
- **R** is a programming language and software environment for...
  - Data tidying.
  - Data visualization.
  - Statistical and econometric analysis.
- Install R from https://cran.r-project.org/

Basic R Programming Concepts

Code instructions written in a programming language.

- **Script** a .R text file with many lines of code.
- **Comment** code lines ignored by the computer, used to explain or annotate your work. Comments start with #.
- **Object** a piece of information (like a number, or dataset) stored in memory that can be used and modified by your code.
- **Function** a reusable "recipe" that takes some objects, performs a specific task and returns an output.

```r
# This is a comment

# Create object "number"

number <- 5

# Define the function "square()"

square <- function(x) { return(x^2) }

# Run function on object, store as "result"

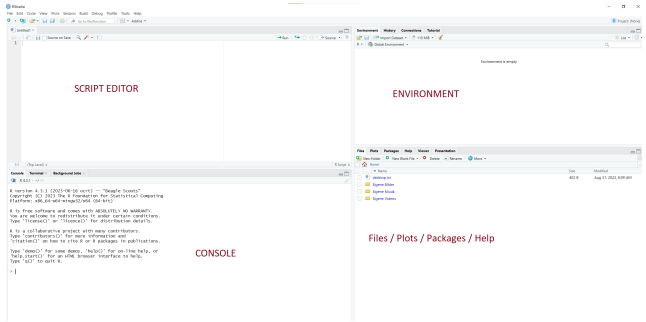
result <- square(number)

# Print the result in the console

print(result)
```

What is RStudio?

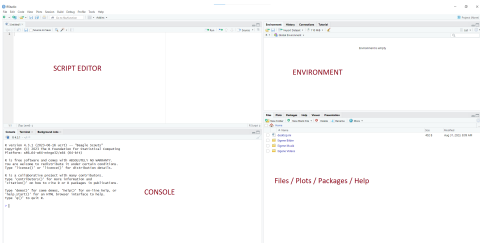
- **RStudio** a workspace that provides a user-friendly interface to write and run code.
- Install RStudio from https://posit.co/download/rstudio-desktop/
- R Studio interface consists of 4 main windows (panes):



RStudio Interface

Console

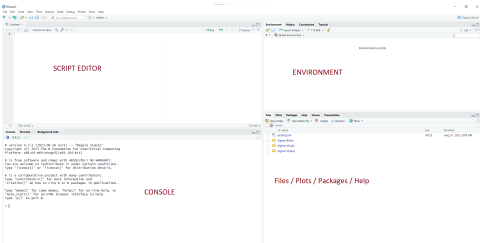
- Output of your coding and computations appears here (exception: graphs).
- It is possible to write code directly into the console.
- Do not use the console to make changes to the data.



RStudio Interface

Script Editor

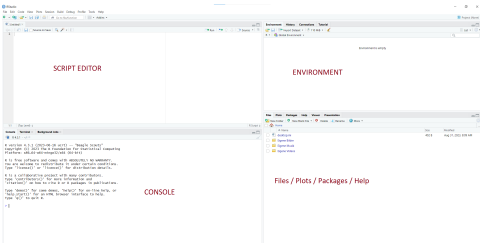
- Open a new R script: click *File* → *New File* → *R Script*.
- Always write your main code in a script: allows you to save and keep track of your work, and to re-run the analysis or make modifications later.
- To send the code in your script to the console, click *run*.



RStudio Interface

Environment

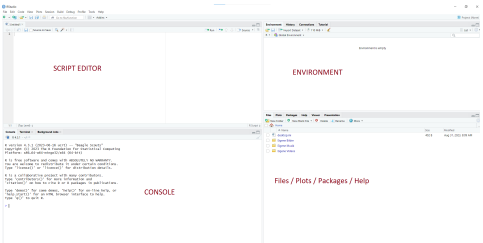
- list of all objects you created
- second pane provides *History* of commands entered into the console



RStudio Interface

Files / Plots / Packages / Help / Viewer window

- *Plots* you create appear here.
- if you search for *Help*, information appears here.



R Packages

What are packages?

- Collections of functions, data, and documentation that extend R's functionality.
- Thousands are available on CRAN, Bioconductor, GitHub, etc.
- Example: `ggplot2` for data visualization.

How to install a package

```
# Install once (downloads from CRAN)
install.packages("ggplot2")
```

How load a package

```
# Load every time you start R
library(ggplot2)
```

Script Set Up

Working directory is the default folder on your computer where R reads and saves files, unless you tell it otherwise.

- Create a new folder on your desktop called "AE_2026" to use as your working directory.
- show current working directory `getwd()`
- setting new working directory to newly created folder `setwd("C:/Users/.../AE_2026")`
- list all files in your working directory `dir()`

```
# Check current working directory
getwd()

# Set a new working directory.
setwd("C:/Users/YourName/Desktop/AE_2026")

# Verify the change
getwd()

# List files in the working directory
dir()
```

Script Set Up

Working directory is the default folder on your computer where R reads and saves files, unless you tell it otherwise.

- Create a new folder on your desktop called "AE_2026" to use as your working directory.
- show current working directory `getwd()`
- setting new working directory to newly created folder `setwd("C:/Users/.../AE_2026")`
- list all files in your working directory `dir()`

```
# Check current working directory
getwd()

# Set a new working directory.
setwd("C:/Users/YourName/Desktop/AE_2026")

# Verify the change
getwd()

# List files in the working directory
dir()
```

R Objects

Object a piece of information (like a number, word, vector, or dataset) stored in memory that can be used and modified by your code.

- create an object: `object <- ...`
- print object to the console: `object`
- identify format using `str(object)`
- get basic information on the object: `class(object)`, `dim(object)`
- list all objects: `ls()`
- remove an object: `rm(object)`

```r
# generate objects
n <- 100 # Scalar object
id <- 1:n # Vector object
age <- sample(18:65, n, replace = TRUE)

# Visualize
age # Print object in console
str(age) # Object type, size
class(age) # Object type
dim(age)
ls() # List all objects
```

R Objects

Object a piece of information (like a number, word, vector, or dataset) stored in memory that can be used and modified by your code.

- create an object: `object <- ...`
- print object to the console: `object`
- identify format using `str(object)`
- get basic information on the object: `class(object)`, `dim(object)`
- list all objects: `ls()`
- remove an object: `rm(object)`

```r
# generate objects
n <- 100 # Scalar object
id <- 1:n # Vector object
age <- sample(18:65, n, replace = TRUE)

# Visualize
age # Print object in console
str(age) # Object type, size
class(age) # Object type
dim(age)
ls() # List all objects
```

Vectors

Vector an ordered sequence of values of the same type (all numbers, or all words).

- `AirPassengers`: number of airline passengers (in thousands) in each month from 1949 to 1960.
- number of elements: `length(vector)`
- access elements by their position, in square brackets: `vector[...]`
- several vectors of the same length can be put together as the columns of a dataset: `data.frame(...)`

```
# Three vectors with one value per month
passengers <- as.numeric(AirPassengers)
year       <- as.numeric(floor(time(AirPassengers)))
month      <- as.numeric(cycle(AirPassengers))

length(passengers)     # 144

# Access elements of a vector
passengers[1]          # first element: 112
passengers[1:3]        # elements 1 to 3: 112 118 132
passengers[c(1, 144)]  # first and last: 112 432

# Create a dataset from the three vectors
df_air <- data.frame(year, month, passengers)
dim(df_air)            # 144 rows, 3 columns
```

Vectors

Vector an ordered sequence of values of the same type (all numbers, or all words).

- `AirPassengers`: number of airline passengers (in thousands) in each month from 1949 to 1960.
- number of elements: `length(vector)`
- access elements by their position, in square brackets: `vector[...]`
- several vectors of the same length can be put together as the columns of a dataset: `data.frame(...)`

```
# Three vectors with one value per month
passengers <- as.numeric(AirPassengers)
year       <- as.numeric(floor(time(AirPassengers)))
month      <- as.numeric(cycle(AirPassengers))

length(passengers)     # 144

# Access elements of a vector
passengers[1]          # first element: 112
passengers[1:3]        # elements 1 to 3: 112 118 132
passengers[c(1, 144)]  # first and last: 112 432

# Create a dataset from the three vectors
df_air <- data.frame(year, month, passengers)
dim(df_air)            # 144 rows, 3 columns
```

Vectors

Vector an ordered sequence of values of the same type (all numbers, or all words).

- `AirPassengers`: number of airline passengers (in thousands) in each month from 1949 to 1960.
- number of elements: `length(vector)`
- access elements by their position, in square brackets: `vector[...]`
- several vectors of the same length can be put together as the columns of a dataset: `data.frame(...)`

```
# Three vectors with one value per month
passengers <- as.numeric(AirPassengers)
year       <- as.numeric(floor(time(AirPassengers)))
month      <- as.numeric(cycle(AirPassengers))

length(passengers)     # 144

# Access elements of a vector
passengers[1]          # first element: 112
passengers[1:3]        # elements 1 to 3: 112 118 132
passengers[c(1, 144)]  # first and last: 112 432

# Create a dataset from the three vectors
df_air <- data.frame(year, month, passengers)
dim(df_air)            # 144 rows, 3 columns
```

From Vectors to a Dataframes

Dataframes are two-dimensional objects that store data in a tabular format, with rows and columns:

- Rows: units of observation (individuals, years, ...).
- Columns: variables (e.g. income, education, GNP, ...) [vectors]

`head(...)` prints the first six rows of the dataset we created from the three vectors:

```
> head(df_air)
  year month passengers
1 1949     1        112
2 1949     2        118
3 1949     3        132
4 1949     4        129
5 1949     5        121
6 1949     6        135
```

a column is a **variable**: the vector `passengers`

a row is an **observation**: March 1949

	year	month	passengers
1	1949	1	112
2	1949	2	118
3	1949	3	132
4	1949	4	129
5	1949	5	121
6	1949	6	135

From Vectors to a Dataframes

Dataframes are two-dimensional objects that store data in a tabular format, with rows and columns:

- Rows: units of observation (individuals, years, ...).
- Columns: variables (e.g. income, education, GNP, ...) [vectors]

`head(...)` prints the first six rows of the dataset we created from the three vectors:

```
> head(df_air)
  year month passengers
1 1949     1        112
2 1949     2        118
3 1949     3        132
4 1949     4        129
5 1949     5        121
6 1949     6        135
```

a column is a **variable**: the vector `passengers`

	year	month	passengers
1	1949	1	112
2	1949	2	118
3	1949	3	132
4	1949	4	129
5	1949	5	121
6	1949	6	135

a row is an **observation**: March 1949

Multiple Data Frames

- R is able to work with multiple datasets at the same time.
- Let us load the built in datasets `longley` dataset.

```
# longley: US macroeconomic data, one row per year (1947-1962)
data(longley)
class(longley)        # "data.frame"
head(longley)
```

```
> head(longley)
     GNP.deflator     GNP Unemployed Armed.Forces Population Year Employed
1947         83.0 234.289      235.6        159.0    107.608 1947   60.323
1948         88.5 259.426      232.5        145.6    108.632 1948   61.122
1949         88.2 258.054      368.2        161.6    109.773 1949   60.171
1950         89.5 284.599      335.1        165.0    110.929 1950   61.187
1951         96.2 328.975      209.9        309.9    112.075 1951   63.221
1952         98.1 346.999      193.2        359.4    113.270 1952   63.639
```

- Each **row** is one year: the unit of observation is the US economy in a given year.
- Each **column** is a variable: here, all seven are numerical.
- The first column has no name: it shows the **row names**, not a variable.

Multiple Data Frames

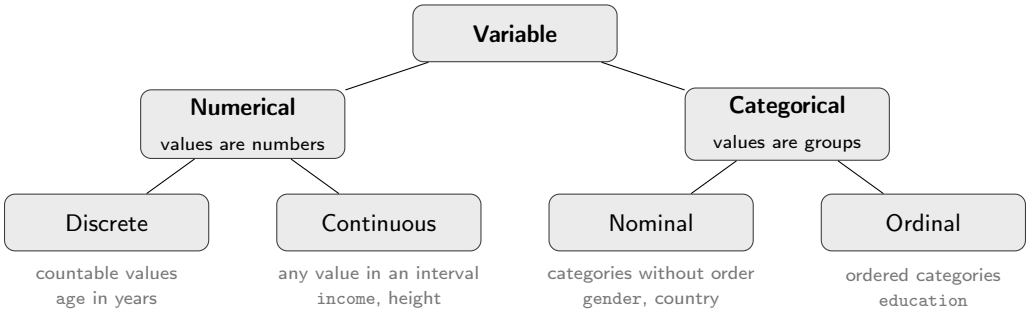
- R is able to work with multiple datasets at the same time.
- Let us load the built in datasets `longley` dataset.

```
# longley: US macroeconomic data, one row per year (1947-1962)
data(longley)
class(longley)        # "data.frame"
head(longley)
```

```
> head(longley)
     GNP.deflator     GNP Unemployed Armed.Forces Population Year Employed
1947         83.0 234.289      235.6        159.0    107.608 1947   60.323
1948         88.5 259.426      232.5        145.6    108.632 1948   61.122
1949         88.2 258.054      368.2        161.6    109.773 1949   60.171
1950         89.5 284.599      335.1        165.0    110.929 1950   61.187
1951         96.2 328.975      209.9        309.9    112.075 1951   63.221
1952         98.1 346.999      193.2        359.4    113.270 1952   63.639
```

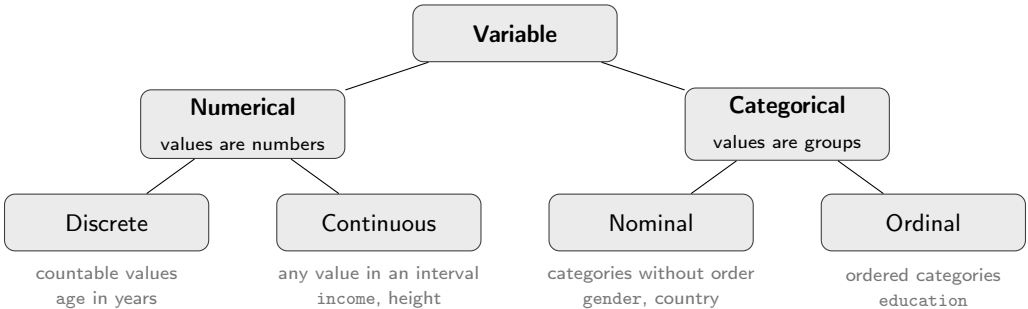
- Each **row** is one year: the unit of observation is the US economy in a given year.
- Each **column** is a variable: here, all seven are numerical.
- The first column has no name: it shows the **row names**, not a variable.

Types of Variables in a Dataset



- A **binary** (dummy) variable is a categorical variable with two categories, coded as 0 and 1: `employed`.
- In R, `class(df$income)` returns the type of a variable: `numeric` or `integer` for numerical variables, `character` or `factor` for categorical ones.

Types of Variables in a Dataset



- A **binary** (dummy) variable is a categorical variable with two categories, coded as 0 and 1: `employed`.
- In R, `class(df$income)` returns the type of a variable: `numeric` or `integer` for numerical variables, `character` or `factor` for categorical ones.

Importing and Saving Data

- .Rdata

```
load("filename.RData"),
save(object, file = "filename.RData")
```

- .csv

```
read.csv("filename.csv"),
write.csv(object, file = "filename.csv")
```

- .xls `read_xls("filename.xls")` from package *readxl*
- .dta `read_dta("filename.xls")` from package *haven*

Inspecting Data

- names of variables in your data: `names(...)`
- list the first/last six observations: `head(...)`, `tail(...)`
- summary statistics (mean, max, min and quartiles; variance, st. dev.): `summary(...)`, `var(...)`, `sd(...)`
- frequency table of one variable and cross-table of two variables: `table(...)`, `table(...,...)`
- check for missing data: `is.na(...)`

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Selecting Observations and Variables

select a specific variable: `dataframe$var1` or `dataframe[[1]]`

```
df$age # Print the age column of df
df[[2]] # Same as previous line
```

select values in specific rows and columns with indices: `dataframe[..., ...]`

```
# Accessing data through row/column indexes
df[1:3,] # Rows 1 to 3, all columns.
df[,c(1,4)] # All rows, Columns 1 and 4.
df[c(1,25),c(1,3)] # Rows 1 and 25, Columns 1 and 3.
```

`dataframe <- dataframe[...,...]`

→ Danger! drops all observations and variables that are not selected from the data.

`dataframe_new <- dataframe[...,...]`
→ stores the desired subset in a new dataframe, while keeping the original data.

```
df_first_fifty_rows <- df[1:50,]
```

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Accessing data - Logical Operators

R can use **logical conditions** to generate logical vectors:

```
# Logical condition vector
index_old_males <- (df$gender == "Male") & (df$age > 50)
table(index_old_males)
```

We can use the logical vector to select the subsample of the dataset:

```
df_old_males <- df[index_old_males, ]
```

Or, directly:

```
df_old_males <- df[(df$gender == "Male") & (df$age > 50), ]
```

Ask ChatGPT: "can you list me the most common logical operations in R"

Transforming variables in a Data Frame

To drop *var1* use:

```
dataframe$var1 <- NULL
```

Generating new variables as transformation of existing variables, e.g.:

```
# New variables generation
df$log_age <- log(df$age)
df$is_higly_educated <- df$education %in% c("Master", "PhD")
```

The variable type of `is_higly_educated` is...

Transforming variables in a Data Frame

To drop *var1* use:

```
dataframe$var1 <- NULL
```

Generating new variables as transformation of existing variables, e.g.:

```
# New variables generation
df$log_age <- log(df$age)
df$is_higly_educated <- df$education %in% c("Master", "PhD")
```

The variable type of `is_higly_educated` is...

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Plotting Data with ggplot2

```
install.packages("ggplot2")
library(ggplot2)
```

```
# Histogram of income
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 30) +
  labs(x = "Income", y = "Frequency")
```

```
# Scatterplot of income against age
ggplot(df, aes(x = income, y = age)) +
  geom_point(size  = 2) +
  labs(x = "Income", y = "Age")
```

Main Types of Graphs in Economic Research

Which graph to use depends on **how many** variables you want to show and on their **type**:

- **Continuous** variables take many numerical values (income, age, GDP).
- **Categorical** variables take a few values or labels (gender, education, country).

Graph	Variables	Shows	ggplot2
Histogram	1 continuous	distribution of a variable	`geom_histogram()`
Bar chart	1 categorical	number of observations per category	`geom_bar()`
Box plot	1 continuous + 1 categorical	distribution within each category	`geom_boxplot()`
Scatter plot	2 continuous	relationship between two variables	`geom_point()`
Line graph	1 continuous + time	evolution of a variable over time	`geom_line()`

All of them allow for **different categories**: a categorical variable can be added as a colour, a fill, or a separate panel.

Main Types of Graphs in Economic Research

Which graph to use depends on **how many** variables you want to show and on their **type**:

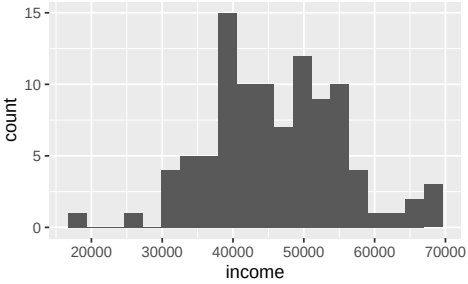
- **Continuous** variables take many numerical values (income, age, GDP).
- **Categorical** variables take a few values or labels (gender, education, country).

Graph	Variables	Shows	ggplot2
Histogram	1 continuous	distribution of a variable	`geom_histogram()`
Bar chart	1 categorical	number of observations per category	`geom_bar()`
Box plot	1 continuous + 1 categorical	distribution within each category	`geom_boxplot()`
Scatter plot	2 continuous	relationship between two variables	`geom_point()`
Line graph	1 continuous + time	evolution of a variable over time	`geom_line()`

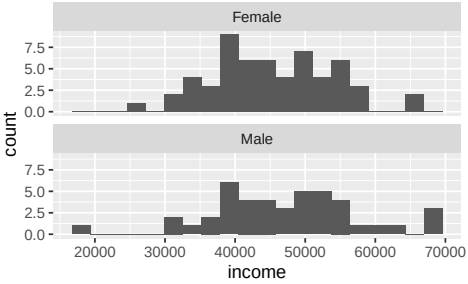
All of them allow for **different categories**: a categorical variable can be added as a colour, a fill, or a separate panel.

Histogram

One continuous variable. Each bar counts the observations in an interval (bin). Categories: one panel per category.



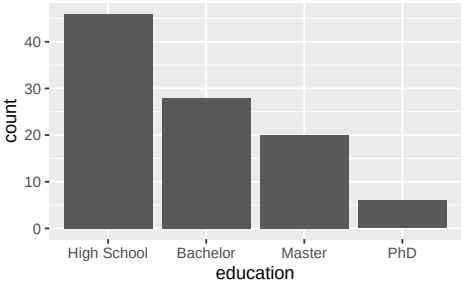

```
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 20)
```

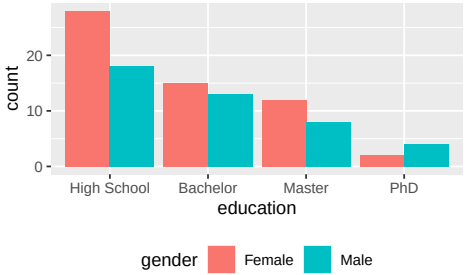



```
ggplot(df, aes(x = income)) +
  geom_histogram(bins = 20) +
  facet_wrap(~ gender, ncol = 1)
```

Bar Chart

One categorical variable. Each bar counts the observations in a category. Categories: one bar per category.



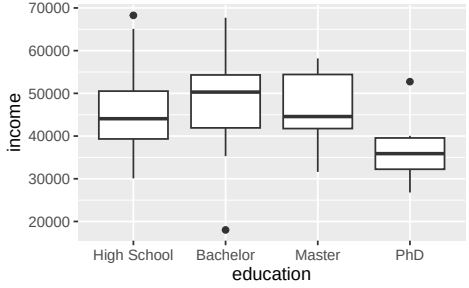


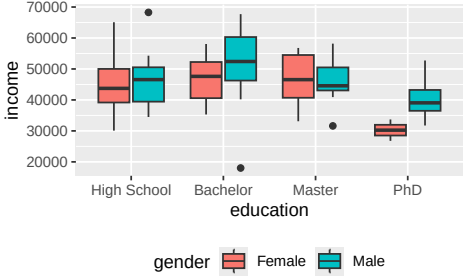

```
ggplot(df, aes(x = education)) +
  geom_bar()
```

```
ggplot(df, aes(x = education, fill = gender)) +
  geom_bar(position = "dodge") +
  theme(legend.position = "bottom")
```

Box Plot

One continuous and one categorical variable. Each box shows the median and the quartiles of a category. Categories: split each box by a second categorical variable.



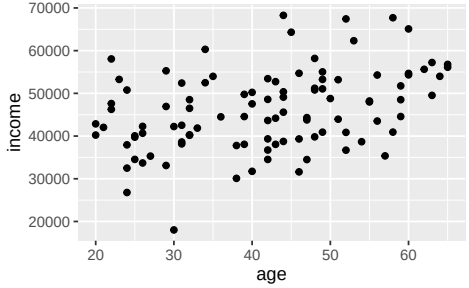



```
ggplot(df, aes(x = education, y = income)) +
  geom_boxplot()
```

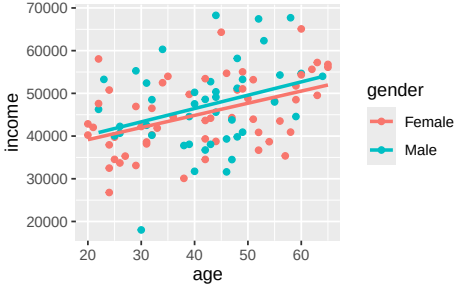
```
ggplot(df, aes(x = education, y = income,
               fill = gender)) +
  geom_boxplot() +
  theme(legend.position = "bottom")
```

Scatter Plot

Two continuous variables. Each point is one observation. Categories: one colour (and one fitted line) per category.



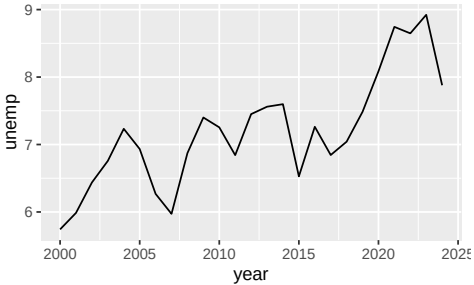

```
ggplot(df, aes(x = age, y = income)) +
  geom_point()
```




```
ggplot(df, aes(x = age, y = income,
               colour = gender)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

Line Graph

One continuous variable over time. Points are connected in time order. Categories: one line per category.






```
ggplot(df_ts[df_ts$country == "A", ],
       aes(x = year, y = unemp)) +
  geom_line()
```

```
ggplot(df_ts, aes(x = year, y = unemp,
                  colour = country)) +
  geom_line()
```

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).